

```

"""
JARVIS - Assistente personale per Windows
=====

Interagisce a voce o a testo, apre
programmi/file, cerca file sul PC
e organizza cartelle spostando i file per
tipo.

INSTALLAZIONE (apri il Prompt dei comandi /
PowerShell e lancia):
    pip install speechrecognition pyttsx3
pyaudio

NOTA su pyaudio: su Windows a volte "pip
install pyaudio" fallisce.
Se succede, scarica il file .whl adatto da:

https://www.lfd.uci.edu/~gohlke/pythonlibs/
#pyaudio
e installalo con: pip install nome_file.whl

AVVIO:
    python jarvis.py
"""

import os
import sys
import subprocess
import shutil
import webbrowser

```

```

from pathlib import Path

# ----- Voce (opzionale: se le
# librerie non sono installate, si passa solo
# al testo) -----
VOCE_DISPONIBILE = True
try:
    import speech_recognition as sr
    import pyttsx3
except ImportError:
    VOCE_DISPONIBILE = False

# ----- Configurazione -----
# Programmi comuni di Windows: aggiungi qui
# i tuoi preferiti.
# La chiave è quello che dirai/scriverai,
# il valore è il comando per aprirlo.
PROGRAMMI = {
    "blocco note": "notepad.exe",
    "notepad": "notepad.exe",
    "calcolatrice": "calc.exe",
    "paint": "mspaint.exe",
    "esplora file": "explorer.exe",
    "cmd": "cmd.exe",
    "powershell": "powershell.exe",
    "word": "winword.exe",
    "excel": "excel.exe",
    "chrome": "chrome.exe",
    "edge": "msedge.exe",
}

# Cartelle di ricerca predefinite per
# l'organizzazione file
ESTENSIONI = {

```

```

    "immagini": [".jpg", ".jpeg", ".png",
".gif", ".bmp", ".webp"],
    "documenti": [".pdf", ".docx", ".doc",
".txt", ".xlsx", ".xls", ".pptx", ".ppt"],
    "video": [".mp4", ".mkv", ".avi",
".mov"],
    "musica": [".mp3", ".wav", ".flac"],
    "archivi": [".zip", ".rar", ".7z"],
    "installer": [".exe", ".msi"],
}

```

```

class Jarvis:
    def __init__(self):
        self.usa_voce = False
        if VOCE_DISPONIBILE:
            try:
                self.tts = pyttsx3.init()

self.tts.setProperty("rate", 175)
        self.recognizer =
sr.Recognizer()
        self.usa_voce = True
    except Exception:
        self.usa_voce = False

    # ----- Output -----
    def parla(self, testo):
        print(f"Jarvis: {testo}")
        if self.usa_voce:
            try:
                self.tts.say(testo)
                self.tts.runAndWait()
            except Exception:

```

```

pass

# ----- Input vocale -----
def ascolta(self):
    with sr.Microphone() as source:
        print("(In ascolto... parla pure)")

self.recognizer.adjust_for_ambient_noise(source, duration=0.5)
    try:
        audio =
self.recognizer.listen(source, timeout=6,
phrase_time_limit=8)
    except sr.WaitTimeoutError:
        return ""
    try:
        testo =
self.recognizer.recognize_google(audio,
language="it-IT")
        print(f"Tu: {testo}")
        return testo.lower()
    except (sr.UnknownValueError,
sr.RequestError):
        return ""

# ----- Azioni -----
def apri_programma(self, nome):
    nome = nome.strip().lower()
    for chiave, comando in
PROGRAMMI.items():
        if chiave in nome:
            try:

```

```

subprocess.Popen(comando)
                self.parla(f"Apro
{chiave}.")
                except Exception as e:
                    self.parla(f"Non sono
riuscito ad aprire {chiave}: {e}")
                    return True
                return False

    def apri_percorso(self, percorso):
        percorso =
percorso.strip().strip('')
        p = Path(percorso).expanduser()
        if p.exists():
            os.startfile(p)
            self.parla(f"Apro {p.name}.")
        else:
            self.parla("Non trovo quel file
o quella cartella.")

    def cerca_file(self, nome_file,
cartella_base=None):
        cartella_base = Path(cartella_base)
        if cartella_base else Path.home()
            self.parla(f"Cerco '{nome_file}'
dentro {cartella_base}, un attimo...")
            trovati = []
            for root, dirs, files in
os.walk(cartella_base):
                for f in files:
                    if nome_file.lower() in
f.lower():
trovati.append(os.path.join(root, f))

```

```

        if len(trovati) >= 15:
            break
        if trovati:
            self.parla(f"Ho trovato
{len(trovati)} risultati:")
            for t in trovati[:15]:
                print(f" - {t}")
        else:
            self.parla("Non ho trovato
nulla con quel nome.")
            return trovati

    def organizza_cartella(self, cartella):
        cartella =
Path(cartella.strip().strip('')).expanduse
r()
        if not cartella.exists() or not
cartella.is_dir():
            self.parla("Quella cartella non
esiste.")
            return

        spostati = 0
        for file in cartella.iterdir():
            if file.is_dir():
                continue
            ext = file.suffix.lower()
            categoria = None
            for cat, estensioni in
ESTENSIONI.items():
                if ext in estensioni:
                    categoria = cat
                    break
            categoria = categoria or

```

```

"altro"

        dest_dir = cartella / categoria
        dest_dir.mkdir(exist_ok=True)
        try:
            shutil.move(str(file),
str(dest_dir / file.name))
            spostati += 1
        except Exception:
            pass

        self.parla(f"Fatto. Ho organizzato
{spostati} file in {cartella}.")

    def cerca_sul_web(self, query):
        url =
f"https://www.google.com/search?q=
{query.replace(' ', '+')}}"
        webbrowser.open(url)
        self.parla(f"Ecco cosa ho trovato
per '{query}' sul browser.")

    # ----- Interpretazione comandi --
    -----
    def esegui_comando(self, comando):
        comando = comando.strip().lower()

        if not comando:
            return True

        if comando in ("esci", "stop",
"basta cosi", "chiudi jarvis"):
            self.parla("A presto!")
            return False

```

```

        if comando.startswith("apri "):
            resto = comando[5:].strip()
            # Se sembra un percorso
            (contiene \ o /), aprilo come file/cartella
            if "\\\" in resto or "/" in
            resto or ":" in resto:
                self.apri_percorso(resto)
            elif not
            self.apri_programma(resto):
                self.parla(f"Non conosco il
            programma '{resto}'. Puoi aggiungerlo nella
            lista PROGRAMMI dello script.")
                return True

        if comando.startswith("cerca file
        "):
            nome = comando[len("cerca file
            "):].strip()
            self.cerca_file(nome)
            return True

        if comando.startswith("cerca "):
            query = comando[len("cerca
            "):].strip()
            self.cerca_sul_web(query)
            return True

        if comando.startswith("organizza
        "):
            cartella =
            comando[len("organizza "):].strip()

            self.organizza_cartella(cartella)

```



```

        return True

        if comando in ("aiuto", "help",
"comandi"):
            self.mostra_aiuto()
            return True

        self.parla("Non ho capito. Scrivi
'aiuto' per vedere i comandi disponibili.")
        return True

    def mostra_aiuto(self):
        aiuto = """
Comandi disponibili:
    apri <programma>          -> es: apri
blocco note / apri chrome
    apri <percorso completo> -> es: apri
C:\\Users\\Nome\\Documenti\\file.pdf
    cerca file <nome>         -> cerca un
file sul PC (parte dalla cartella utente)
    cerca <query>             -> cerca
qualcosa su Google
    organizza <cartella>      -> smista i
file di quella cartella per tipo
                                es:
organizza C:\\Users\\Nome\\Downloads
    aiuto                     -> mostra
questo elenco
    esci                       -> chiude
Jarvis
"""

        print(aiuto)

    # ----- Loop principale -----

```

```

def avvia(self):
    self.parla("Ciao, sono Jarvis. Sono
pronto ad aiutarti.")
    self.mostra_aiuto()

    modalita_voce = False
    if self.usa_voce:
        scelta = input("Vuoi usare la
voce? (s/n): ").strip().lower()
        modalita_voce = scelta == "s"
    else:
        print("(Librerie voce non
installate: procedo solo a testo. Vedi le
istruzioni in cima al file.)")

    attivo = True
    while attivo:
        if modalita_voce:
            comando = self.ascolta()
            if not comando:
                continue
        else:
            comando = input("\nTu: ")

        attivo =
self.esegui_comando(comando)

if __name__ == "__main__":
    jarvis = Jarvis()
    jarvis.avvia()

```

